claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_134043cd-8c3\agent-a029caacf974b59a0.jsonl`
- SHA-256: `eb9bed8cff2029f3fc11fb3c410c1e0e8b9ccc214ae36c1df91c1b166100a7c5`
- Source modified: `2026-06-20T09:01:51+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_134043cd-8c3`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T08:59:23.207Z)

IMPORTANT grounding (verified this session): the engine is now DECOUPLED ? pluggable StorageBackend (local + S3/GCS/DO Spaces) in backend/storage/, and a worker `python -m backend.worker {infer|train}` (backend/worker/__main__.py) that pulls from storage -> runs the pipeline -> pushes results. Validated e2e on the CPU droplet 168.144.126.176 reading/writing DO Spaces with the GPU MOCKED (stub detector, RALLY_DETECTOR_IMPL=stub). BOTH the khelsutra site (/srv/khelsutra-site) AND the engine (~/rally) already live on that ONE droplet. The droplet is CPU-only (no GPU) and a SHARED testbed (MinIO 9000/9001, GCS emu 9023) ? do not propose touching those. Real GPU WASB (NativeWasbRunner, "M3.5") is the NEXT build, not done. The FastAPI has NO auth (CORS '*'). EXCLUDE any path under .claude/worktrees/ ? those are stale copies; read canonical docs/ + backend/ only.

Map how the engine FastAPI drives a video end-to-end (cwd = C:\\Users\\avidu\\Projects\\badminton-highlight-indexer). Read backend/main.py (process/jobs/query/compile/highlights), backend/api/uploads.py (chunked upload), backend/api/job_worker.py (the in-process async worker), and config.json + backend/config/ (segmenter/detector selection). Report the full upload->process->index->reel flow, every relevant endpoint, the in-process job worker behavior on a single box, how the segmenter/detector is chosen and what each needs (GPU? Gemini key? stub?), and the GAPS for a UI to drive the FULL loop. Quote path:line. Return the schema.

2. assistant (2026-06-20T08:59:26.338Z)

I'll map the engine FastAPI flow end-to-end. Let me start by pulling the repo fresh and reading the key files.

3. user (2026-06-20T08:59:31.077Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-20T08:59:31.110Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
```

```
    ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
    inside the functions also avoids the circular import at module load (main.py imports
this
    module; this module must not import main at import time).
    """
    import logging
    import threading
    import traceback
    from concurrent.futures import ThreadPoolExecutor
    from typing import Any, Dict, Optional

    logger = logging.getLogger(__name__)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
        )


```

```
    def _run_claimed_job(job: Dict[str, Any]) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main
        db_instance = _main.db_instance
        job_id = job["id"]
        req = _main._job_to_request(job)
        try:
            result = _main._execute_processing(
                req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
            if result.get("video_id"):
                db_instance.set_job_video_id(job_id, result["video_id"])
            db_instance.mark_job_done(job_id, result)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _drain_due_jobs() -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main
        ran = 0
        while True:
            job = _main.db_instance.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting()
        return ran


    def _schedule_next_drain_if_waiting() -> None:
        """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
        self-scheduled job resumes with no further client submit. The drain itself re-checks
```

```
127         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128         import backend.main as _main
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return  # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153
```

5. user (2026-06-20T08:59:31.110Z)

```
1       {
2           "sport": "badminton",
3           "ai_provider": "gemini",
4           "ai_model": "gemini-flash-latest",
5           "validation": {
6               "min_resolution_width": 1280,
7               "min_resolution_height": 720,
8               "min_fps": 29.9,
9               "min_blur_threshold": 20.0,
10              "max_scene_cuts_per_minute": 0.5,
11              "relevance": {
12                  "court_green_lower": [
13                      35,
14                      40,
15                      40
16                  ],
17                  "court_green_upper": [
18                      85,
19                      255,
20                      255
21                  ],
22                  "court_pixels_min_ratio": 0.05
23              }
24          },
25          "indexing": {
26              "default_segmenter": "gemini",
27              "use_gpu": true,
28              "motion_threshold": 0.08,
29              "min_segment_duration": 3.0,
```

```
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "detector_model": "fasterrcnn_resnet50_fpn",
            "detector_score_threshold": 0.5,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_prefetch_workers": 2,
            "min_action_window_duration": 2.0,
            "log_yolo_candidates": true,
            "store_yolo_candidates": true,
            "ai_handoff_padding": 2.0,
            "ai_max_workers": 1,
            "tracknet": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/TrackNetV4",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "TrackNetV4",
                "weights_path": "",
                "queue_length": 5,
                "stage_in_wsl": true,
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            },
            "wasb": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/WASB-SBDT",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "wasb",
                "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
                "sport": "badminton",
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            }
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1,
            "watermark": {
                "enabled": true,
                "text": "Generated by Khelsutra.guru",
                "position": "bottom-right",
                "opacity": 0.85,
                "logo_path": "",
                "font_path": ""
            }
        }
    }
```

6. user (2026-06-20T08:59:31.110Z)

```
    """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.

    Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
    proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
```

```
    under security.upload_dir, then submitted to /api/process by the returned path.
    Upload state is in-process by design (this is the single-box alpha): a server restart
    aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    arrives with PR-3 identity scoping.

    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
    reads the live value the server set at startup (and so tests that monkeypatch
    ``backend.main.global_config`` flow through unchanged).
    """
    import os
    import threading
    import uuid
    from typing import Any, Dict, Optional

    from fastapi import APIRouter, HTTPException, Request
    from pydantic import BaseModel

    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
        import backend.main as _main
        sec = getattr(_main.global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
```

```
69              name = safe_output_filename(req.filename)
70          except ValueError as e:
71              raise HTTPException(status_code=400, detail=str(e)) from e
72          ext = os.path.splitext(name)[1].lower().lstrip(".")
73          if ext not in exts:
74              raise HTTPException(status_code=400,
75                                  detail=f"Extension '.{ext}' not allowed. Allowed:
{sorted(exts)}")
76          if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77              raise HTTPException(status_code=413,
78                                  detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79          os.makedirs(upload_dir, exist_ok=True)
80          upload_id = uuid.uuid4().hex
81          tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82          open(tmp_path, "wb").close()
83          with _uploads_lock:
84              _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85          return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86                  "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89      @router.put("/api/upload/{upload_id}/part/{index}")
90      async def upload_part(upload_id: str, index: int, request: Request):
91          """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92          _, max_bytes, part_bytes, _ = _upload_cfg()
93          with _uploads_lock:
94              st = _uploads.get(upload_id)
95          if st is None:
96              raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97          if index != st["parts"]:
98              raise HTTPException(status_code=409,
99                                  detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101         received = 0
102         overflow = None
103         with open(st["tmp_path"], "ab") as fh:
104             async for chunk in request.stream():
105                 received += len(chunk)
106                 if received > part_bytes:
107                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                     break
109                 if st["bytes"] + received > max_bytes:
110                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                     break
112                 fh.write(chunk)
113         if overflow:
114             _abort_upload(upload_id)
115             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117         with _uploads_lock:
118             st["parts"] += 1
119             st["bytes"] += received
120         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
```

```
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129         if st is None:
130             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131         if st["parts"] == 0 or st["parts"] != req.total_parts:
132             raise HTTPException(status_code=409,
133                                 detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134         final_path = os.path.join(upload_dir, st["filename"])
135         if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
136             stem, ext = os.path.splitext(st["filename"])
137             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138         os.replace(st["tmp_path"], final_path)
139         with _uploads_lock:
140             _uploads.pop(upload_id, None)
141         video_id = compute_video_id(final_path)
142         return {"path": os.path.abspath(final_path), "video_id": video_id,
143                 "size_bytes": os.path.getsize(final_path),
144                 "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152         if not known:
153             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154         _abort_upload(upload_id)
155         return {"status": "aborted", "upload_id": upload_id}
156
```

7. user (2026-06-20T08:59:31.598Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10
11      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12      logging.basicConfig(
13          level=logging.INFO,
14          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15          datefmt='%H:%M:%S'
16      )
17      logger = logging.getLogger(__name__)
18      from fastapi import FastAPI, HTTPException
19      from fastapi.middleware.cors import CORSMiddleware
20      from pydantic import BaseModel
21      from typing import List, Dict, Any, Optional
22
23      from fastapi.staticfiles import StaticFiles
24      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
25      from backend.pipeline.validators.base import registry
26      from backend.infrastructure.database import Database
27      from backend.pipeline.segmenters.base import segmenter_registry
```

```
28      from backend.pipeline.stitcher.compiler import VideoCompiler
29      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
30      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
31      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
32      from backend.results import Failure  # standardized error/result contract
33      from backend.reporting import emit_indexer_report
34
35      app = FastAPI(title="Sports Highlight Indexer API")
36
37      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
38      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
39      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
40      app.add_middleware(
41          CORSMiddleware,
42          allow_origins=["*"],
43          allow_credentials=False,
44          allow_methods=["*"],
45          allow_headers=["*"],
46      )
47
48      # Serves static frontend files directly (now under api/ per approved structure)
49      os.makedirs("backend/api/static", exist_ok=True)
50      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
51
52      @app.get("/", response_class=HTMLResponse)
53      def serve_index():
54          index_path = "backend/api/static/index.html"
55          if os.path.exists(index_path):
56              with open(index_path, "r", encoding="utf-8") as f:
57                  return f.read()
58          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
59
60      # Global variables initialized at startup
61      db_instance: Optional[Database] = None
62      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
63
64      # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
65      # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
66      # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
67      # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
68      # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
69      # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
70      # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
71      from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
72          JobWaiting,
73          _arm_wake_timer,
74          _drain_due_jobs,
75          _get_executor,
76          _job_to_request,
77          _MAX_DRAIN_WAKE_SEC,
78          _run_claimed_job,
79          _schedule_next_drain_if_waiting,
80      )
81
82      # Legacy thin wrapper for any remaining direct calls during transition.
83      # New code should import load_app_config / AppConfig from backend.config directly.
84      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
85          cfg = load_app_config(config_path)
```

```
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        video_id = compute_video_id(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
```

```
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

```

```
            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

```

```
        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ------------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) --------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
```

```
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "
```

```
f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for _ in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
```

```
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
```

```
            import subprocess
            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts", "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

```

```
554         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
555         # the executor is in-process. Mark them failed so pollers see a terminal state.
556         swept = db_instance.fail_stale_jobs()
557         if swept:
558             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
559 
560         # CLI processing mode (no web UI needed)
561         if args.video_path and not args.server:
562             print(f"[CLI] Processing video file: {args.video_path}")
563             if not os.path.exists(args.video_path):
564                 print(f"[FAIL] Video file not found: {args.video_path}")
565                 return
566 
567             video_id = compute_video_id(args.video_path)
568             was_reingest = db_instance.get_video(video_id) is not None
569 
570             # Validate (with-context variant surfaces ffprobe_meta for the report)
571             print("[CLI] Validating...")
572             val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
573             failures = [r for r in val_results if not r.passed]
574 
575             # Save video status
576             status = "failed" if failures else "processed"
577             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
578 
579             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
580             segmenter_actual = None
581             segmentation_ran = False
582             try:
583                 print("\n" + "="*40)
584                 print("VALIDATION SUMMARY")
585                 print("="*40)
586                 for r in val_results:
587                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
588                     print(f"{status_symbol} {r.validator_name}: {r.message}")
589                 print("="*40 + "\n")
590 
591                 if failures:
592                     print("[FAIL] Video failed checks. Indexing halted.")
593                     return
594 
595                 # Index
596                 segmenter_cls = segmenter_registry.get(seg_name)
597                 if not segmenter_cls:
598                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
599                     return
600 
601                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
602                 segmenter_actual = seg_name
603                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
604                 segmenter = segmenter_cls(db_instance, global_config)
605                 result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
606                 segmentation_ran = True
607 
608                 if isinstance(result, Failure):
609                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
610                     print("[FAIL] Indexing halted for video.")
611                     segments = []
612                 else:
613                     segments = result.value if hasattr(result, "value") else result
```

```
614                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
615                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
616                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
617             finally:
618                 # Always emit the per-video observability report. Never raises.
619                 paths = emit_indexer_report(
620                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
621                     val_results=val_results, val_context=val_context,
622                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
623                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
624                 )
625                 if paths:
626                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
627                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
628                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
629             return
630
631         # Start Local Server Mode
632         if args.server or not args.video_path:
633             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
634             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
635             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
636             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
637             _get_executor().submit(_drain_due_jobs)
638             uvicorn.run(app, host="127.0.0.1", port=args.port)
639
640     if __name__ == "__main__":
641         main()
642
```

8. assistant (2026-06-20T08:59:34.575Z)

Let me read the config module and the segmenter/detector selection internals.

9. user (2026-06-20T08:59:36.256Z)

```
__init__.py
__pycache__
loader.py
models.py
---SEGMENTERS---
__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
```

10. user (2026-06-20T08:59:36.607Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
```

```
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

11. assistant (2026-06-20T08:59:39.194Z)

Let me look at how segmenters get registered, the detector selection (detector_impl/RALLY_DETECTOR_IMPL), and the hybrid segmenters' GPU/Gemini requirements.

12. user (2026-06-20T08:59:40.612Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4       from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7       from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9       __all__ = [
10          "segmenter_registry",
11          "BasePlaySegmenter",
12          "MotionPlaySegmenter",
13          "GeminiPlaySegmenter",
14          "HybridYoloSegmenter",
15          "TrackNetHybridSegmenter",
16          "WasbHybridSegmenter",
17          "FusionSegmenter",
18      ]
19
```

13. user (2026-06-20T08:59:41.549Z)

```
1       import os
2       import time
3       import logging
4       import subprocess
5       import concurrent.futures
6       from typing import List, Dict, Any, Tuple, Callable
7       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8       from backend.utils.video import get_video_duration, extract_video_chunk
9       from backend.sports import sport_registry
10      from backend.providers import provider_registry
11      from backend.reporting import run_signals
12
13      logger = logging.getLogger(__name__)
14
15
16      def _chunk_workdir(video_id: str) -> str:
17          """Per-video chunk staging dir under output/chunks/<video_id>.
18
19          Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20          otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21          in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22          """
23          return os.path.join("output", "chunks", video_id)
24
25
26      def _parse_time(val) -> float:
27          """Converts a timestamp value to total float seconds.
28          Handles float/int pass-through, plain numeric strings, and
29          colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
30          when unexpected formats are encountered."""
31          if isinstance(val, (int, float)):
32              return float(val)
33          if isinstance(val, str):
34              val = val.strip()
35              if ":" in val:
36                  # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
```

```
37                  # Convert it, but warn so the issue is visible in logs.
38                  parts = val.split(":")
39                  parts.reverse()
40                  total = 0.0
41                  for i, p in enumerate(parts):
42                      try:
43                          total += float(p) * (60 ** i)
44                      except ValueError:
45                          pass
46                  logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
47                  return total
48              try:
49                  return float(val)
50              except ValueError:
51                  logger.warning(f"Could not parse timestamp string '{val}' as a number.
Defaulting to 0.0.")
52                  return 0.0
53          logger.warning(f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0.")
54          return 0.0
55
56
57      class GeminiPlaySegmenter(BasePlaySegmenter):
58          @property
59          def name(self) -> str:
60              return "gemini"
61
62          @property
63          def description(self) -> str:
64              return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
65
66          def _resolve_provider(self, model_name: str):
67              """Resolve the sport profile, provider, and API key (the process_video
preamble).
68
69              Returns (sport_profile, provider) on success, or None to signal any of the
70              early-return cases (unknown sport, missing API key, unknown provider) ? the
71              caller returns [] for None.
72              """
73              # Get sport profile config
74              sport_name = self.config.get("sport", "badminton")
75              try:
76                  sport_profile = sport_registry.get(sport_name)
77              except KeyError as e:
78                  logger.error(str(e))
79                  return None
80
```

14. user (2026-06-20T08:59:42.334Z)

```
deploy\digitalocean\README.md:61:`RALLY_DETECTOR_IMPL=stub python -m pytest
tests/test_stub_runner.py -q`.
backend\config\models.py:70:    # --- Linux-native runner only (detector_impl='native'); ignored
by the WSL runner. ---
backend\config\models.py:128:    # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
backend\config\models.py:129:    detector_impl: str = "auto"
backend\worker\__main__.py:9:are byte-for-byte unchanged. Combine with
``RALLY_DETECTOR_IMPL=stub`` and
backend\worker\__main__.py:164:    detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl:
auto=WSL, native=GPU box,
backend\worker\__main__.py:326:                          "(needs videos/ in the corpus; honours
detector_impl).")
```

backend\pipeline\detectors\native_wasb_runner.py:22:the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
backend\pipeline\detectors\native_wasb_runner.py:23:(``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``); ``detector_impl="native"``.
backend\pipeline\detectors\stub_runner.py:20:``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
backend\pipeline\segmenters\wasb_hybrid.py:42:   # seam (imported only when detector_impl='native' is actually selected).
backend\pipeline\segmenters\wasb_hybrid.py:46:"detector_impl": "native", "device": cfg.device}
backend\pipeline\segmenters\fusion_hybrid.py:82:   "detector_impl='native' supports the WASB substrate only ? set "
backend\pipeline\segmenters\fusion_hybrid.py:86:   return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path, "detector_impl": "native",
backend\pipeline\segmenters\trajectory_hybrid.py:61:   f"detector_impl='native' is not supported for the "
backend\pipeline\segmenters\trajectory_hybrid.py:63:   f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
backend\pipeline\segmenters\trajectory_hybrid.py:68:   ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
backend\pipeline\segmenters\trajectory_hybrid.py:76:   impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or "auto").lower()
backend\pipeline\segmenters\trajectory_hybrid.py:79:   logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
backend\pipeline\segmenters\trajectory_hybrid.py:81:   return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl": "stub"}
backend\pipeline\segmenters\trajectory_hybrid.py:83:   logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
backend\pipeline\segmenters\trajectory_hybrid.py:190:   # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
backend\pipeline\segmenters\trajectory_hybrid.py:191:   # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
docs\SETUP_NEW_MACHINE.md:127: "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "stub", "skip_ai_handoff": true },
docs\SETUP_NEW_MACHINE.md:137:env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub \
docs\SETUP_NEW_MACHINE.md:140:Expect: `detector_impl=stub ? CPU mock runner`, `Phase 3 SKIPPED (fully-local)`, and
docs\SETUP_NEW_MACHINE.md:141:`Indexed 2 play segments` in `~/rally/out/sports_indexer.db`.
*(`detector_impl=stub` swaps the GPU+WSL
docs\SETUP_NEW_MACHINE.md:187:env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub python -m backend.worker infer \
docs\SETUP_NEW_MACHINE.md:255: `detector_impl=stub`.
docs\DECOUPLED_COMPUTE\HANDOFF.md:21:| #247 | CPU-mock detector seam (`RALLY_DETECTOR_IMPL=stub` / `indexing.detector_impl=stub`) | `backend/pipeline/detectors/stub_runner.py` |
docs\DECOUPLED_COMPUTE\HANDOFF.md:41: (`storage.s3` sgp1 + `skip_ai_handoff` + `detector_impl=stub` + permissive validation), `/root/do_stage_{labels,videos}.sh`.
docs\DECOUPLED_COMPUTE\HANDOFF.md:59:   `detector_impl` seam (`trajectory_hybrid._resolve_runner`). Scout details: `06-RISKS-AND-OPEN-QUESTIONS.md` (E1?E6 in the
tests\test_native_wasb_runner.py:221:# --- the detector_impl='native' seam (parallels the stub seam) -------------------------
tests\test_native_wasb_runner.py:223:   monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
tests\test_native_wasb_runner.py:225:   runner, params = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
tests\test_native_wasb_runner.py:227:   assert params.get("detector_impl") == "native"
tests\test_native_wasb_runner.py:231:   monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
tests\test_native_wasb_runner.py:233:   runner, _ = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "auto"})
tests\test_native_wasb_runner.py:238:   monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
tests\test_native_wasb_runner.py:240:   runner, params = FusionSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
tests\test_native_wasb_runner.py:246:   monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)

```
tests\test_native_wasb_runner.py:250:            {"detector_impl": "native", "fusion":
{"substrate": "tracknet"}})
tests\test_native_wasb_runner.py:254:    monkeypatch.delenv("RALLY_DETECTOR_IMPL",
raising=False)
tests\test_native_wasb_runner.py:257:        TrackNetHybridSegmenter(None,
{})._resolve_runner({"detector_impl": "native"})
tests\test_stub_runner.py:93:    monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
tests\test_stub_runner.py:99:    monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
tests\test_stub_runner.py:100:    runner, params = seg._resolve_runner({"detector_impl":
"stub"})
tests\test_stub_runner.py:102:    assert params == {"detector_impl": "stub"}
tests\test_stub_runner.py:106:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
tests\test_stub_runner.py:108:    runner, _ = seg._resolve_runner({"detector_impl": "auto"})
tests\test_worker.py:196:    RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the
stub IS the runner)."""
tests\test_worker.py:200:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU
stub runner (no GPU/WSL)
```

15. assistant (2026-06-20T08:59:44.870Z)

Now let me read the detector selection seam in trajectory_hybrid.py and the runner requirements (GPU/WSL/stub).

16. user (2026-06-20T08:59:46.776Z)

```
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47          #: config section under `indexing` (velocity_threshold lives there), the
48          #: output subdir for trajectories, and the metadata detector-tag prefix.
49          family: str = ""
50          #: human-readable label used in log lines.
51          display_label: str = ""
52
53          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54              """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
55              raise NotImplementedError
56
57          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58              """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59              native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60              raise NotImplementedError(
61                  f"detector_impl='native' is not supported for the "
62                  f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63                  f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65          def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66              """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68              ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69              ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
```

```
pipeline
70              is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71              in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72              ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73              (the WSL runner) so the default production path is unchanged
74              (``docs/DECOUPLED_COMPUTE/``).
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77              if impl == "stub":
78                  from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79                  logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                              self.display_label or "trajectory")
81                  return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82              if impl in ("native", "native_wasb", "wasb_native"):
83                  logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                              self.display_label or "trajectory")
85                  return self._build_native_runner(idx_cfg)
86              return self._build_runner(idx_cfg)
87
88          @staticmethod
89          def _probe_fps_width(video_path: str) -> tuple:
90              """Return (fps, frame_width) for a video, with sensible fallbacks."""
91              cap = cv2.VideoCapture(video_path)
92              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94              cap.release()
95              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97          @staticmethod
98          def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99              """Provider-reported exchange count, else a duration-based estimate.
100
101             One canonical implementation ? the twins had drifted (wasb relied on
102             ``int(None)`` raising into the fallback; same result, by accident).
103             """
104             shot_count = segment.get("shuttle_exchange_count")
105             if shot_count is None:
106                 return max(3, int(duration / 0.8))
107             try:
108                 return int(shot_count)
109             except (ValueError, TypeError):
110                 return max(3, int(duration / 0.8))
111
112         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
113         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                     frame_width: float, video_path: str,
115                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119             features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120             return {
121                 "peak_velocity": cw["peak_velocity"],
122                 "mean_velocity": cw["mean_velocity"],
123                 "active_frame_count": cw["active_frame_count"],
124                 "track_id_count": cw["track_id_count"],
```

```
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
            # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
            # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
            # measure it against the Gemini path, or to avoid the cloud entirely). With
            # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
            skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            if skip_ai:
                model_name = "local-noai"
                logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
                            "be emitted as rallies; no %s handoff, no API key needed.",
                            label, provider_name)
            else:
                model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
```

```
181                     return []
182
183             video_duration = get_video_duration(video_path)
184             if video_duration <= 0:
185                 logger.error("Invalid video duration.")
186                 return []
187             fps, frame_width = self._probe_fps_width(video_path)
188
189             # --- Runner config + windowing thresholds ---
190             # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191             # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192             # it delegates to the subclass _build_runner (the real WSL/native runner).
193             runner, runner_params = self._resolve_runner(idx_cfg)
194
195             velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
196             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
197             min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
198             # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
199             max_window_duration = idx_cfg.get("max_window_duration", 0.0)
```

17. user (2026-06-20T08:59:46.943Z)

```
1       """WASB-based hybrid play segmenter.
2
3       Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4       WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5       badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7       For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8       candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9       min_window_duration) are what get tuned against golden rally labels.
10
11      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13      WASB runner and identity.
14      """
15      from typing import Any, Dict, Tuple
16
17      from backend.pipeline.segmenters.base import segmenter_registry
18      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19      from backend.pipeline.detectors.base import DetectorRunner
20      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23      class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24          family = "wasb"
25          display_label = "WASB"
26
27          @property
28          def name(self) -> str:
29              return "wasb_hybrid"
30
31          @property
32          def description(self) -> str:
33              return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                      "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37              cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38              return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
```

```
40          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41              # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42              # seam (imported only when detector_impl='native' is actually selected).
43              from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45              return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                             "detector_impl": "native", "device": cfg.device}
47
48
49      segmenter_registry.register(WasbHybridSegmenter)
50
```

18. assistant (2026-06-20T08:59:50.017Z)

Now let me read the config models and check the static UI to see what exists for driving the loop, plus the database job methods.

19. user (2026-06-20T08:59:50.958Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
```

```
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
73          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
74          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
75          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
76          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
77          keep_frames: bool = False
78          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79          min_free_gb: float = 0.0
80          # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
81          # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
82          # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
83          # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
84          # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
85          stream_video: bool = False
86
87
88      class FusionConfig(BaseModel):
89          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
90
91          Every default reproduces control behaviour: the fusion segmenter persists the
92          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
93          true ? the person-cue features, but it does NOT gate the served handoff unless a
94          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
95          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
```

```
only);
96          supplied ? off-court persons (spectators / adjacent courts) are excluded.
97          """
98          # Which trajectory substrate seeds the windows (shuttle stays primary).
99          substrate: Literal["wasb", "tracknet"] = "wasb"
100         # Windowing velocity threshold for the fusion family (the engine reads
101         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
102         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
103         velocity_threshold: float = 0.02
104         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
105         # enabled ? so the default path never imports torch / touches the GPU.
106         cue_flags: dict[str, bool] = Field(default_factory=dict)
107         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
108         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
109         min_crossings: int = Field(default=0, ge=0)
110         # Served Gate B: when the person cue is on, drop windows with median in-court
players
111         # < this. 0 = OFF.
112         min_players: int = Field(default=0, ge=0)
113         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
114         court_polygon: Optional[list[list[float]]] = None
115         # Net line for the person two-sided-motion split (person features only ? the shuttle
116         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
117         net_axis: Literal["x", "y"] = "y"
118         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
119
120
121     class IndexingConfig(BaseModel):
122         default_segmenter: str = "yolo_hybrid"
123         use_gpu: bool = True
124         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
125         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
126         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
127         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
128         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
129         detector_impl: str = "auto"
130         motion_threshold: float = 0.08
131         min_segment_duration: float = 3.0
132         dead_time_merge_gap: float = 2.0
133         chunk_duration: float = 90.0
134         chunk_overlap: float = 10.0
135         detector_model: str = "fasterrcnn_resnet50_fpn"
136         detector_score_threshold: float = 0.5
137         yolo_velocity_threshold: float = 0.15
138         yolo_frame_skip: int = 3
139         yolo_tracker: str = "bytetrack.yaml"
140         yolo_prefetch_workers: int = 2
141         min_action_window_duration: float = 2.0
142         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
143         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
144         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
145         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
146         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
147         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
148         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
```

```
149         max_window_duration: float = 6.0
150         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
151         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
152         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
153         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
154         # Only cuts (recall can't drop). OFF by default until measured/promoted.
155         window_hard_cap: bool = False
156         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
157         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
158         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
159         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
160         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
161         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
162         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
163         window_inactivity_end: float = 1.5
164         inactivity_rally_thresh: float = 0.20
165         inactivity_min_density: float = 0.30
166         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
167         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
168         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
169         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
170         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
171         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
172         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
173         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
174         window_blob_fallback: bool = False
175         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
176         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
177         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
178         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
179         window_blob_factor: float = 4.0
180         log_yolo_candidates: bool = True
181         store_yolo_candidates: bool = True
182         ai_handoff_padding: float = 2.0
183         ai_max_workers: int = 5
184         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
185         # chunks of a high-bitrate (4K) source blow past the provider upload cap
186         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
187         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
188         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
189         # Matches gemini_refine's --clip-scale-width default.
190         ai_chunk_scale_width: int = 1280
191         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
```

```
192         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
193         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
194         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
195         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
196         skip_ai_handoff: bool = False
197         # Optional debug knob: trim every video to the first N seconds before processing.
198         debug_duration: Optional[float] = None
199
200         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
201         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
202         fusion: FusionConfig = Field(default_factory=FusionConfig)
203
204         @field_validator("default_segmenter")
205         @classmethod
206         def segmenter_must_be_registered(cls, v: str) -> str:
207             # Registry check happens at runtime in main/segmenter selection.
208             # Here we just allow any string for forward compatibility.
209             return v
210
211         @model_validator(mode="after")
212         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
213             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
214             # step makes `while t < duration: t += step` loop forever, growing memory before
any
215             # GPU work. Reject the bad config at load time instead.
216             if self.chunk_overlap >= self.chunk_duration:
217                 raise ValueError(
218                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
219                     f"({self.chunk_duration}); otherwise chunking never advances."
220                 )
221             return self
222
223
224     class OutputConfig(BaseModel):
225         default_highlights_name: str = "highlights.mp4"
226         db_name: str = "sports_indexer.db"
227         # Directory where the DB, highlight reels, and processed clips are written.
228         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
229         output_dir: str = "output"
230
231
232     class WatermarkConfig(BaseModel):
233         """Branding overlay burned into each highlight clip during the per-clip re-encode
234         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
235         (under `stitching.watermark` in config.json) to turn it off. The text is fully
236         configurable. The concat stage stays stream-copy (-c copy)."""
237         enabled: bool = True
238         text: str = "Generated by Khelsutra.guru"
239         logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
240         font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
241         font: str = "Sans"         # fontconfig family used when font_path is empty
242         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
243         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
244         box: bool = True           # faint backing box behind the text for legibility
245         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
246         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
```

```
247
248
249     class StitchingConfig(BaseModel):
250         begin_padding: float = 0.5
251         end_padding: float = 0.5
252         use_cache: bool = False
253         min_shot_count: int = 1
254         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
255         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
256         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
257         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
258         # local detector over-fires and its false positives are mostly SHORT (motion blips,
259         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
260         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
261         min_rally_duration: float = Field(default=0.0, ge=0.0)
262         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
263
264
265     class LoggingConfig(BaseModel):
266         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
267         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
268         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
269         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
270         # them to WARNING; set true to restore full chatter for request-flow debugging.
271         verbose_http: bool = False
272
273
274     class SecurityConfig(BaseModel):
275         # When set, /api/process video_path must resolve under this directory (hosted/tenant
276         # mode). Default None preserves the single-user local 'any local file' workflow.
277         allowed_video_root: Optional[str] = None
278
279         # --- Chunked upload (B2, PR-2) ---
280         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
281         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
282         # contain upload_dir or /api/process will reject the uploaded paths.
283         upload_dir: str = "output/uploads"
284         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
285         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
286         max_upload_mb: int = 4096
287         max_part_mb: int = 95
288         allowed_upload_extensions: List[str] = ["mp4", "mov"]
289
290
291     class StorageConfig(BaseModel):
292         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295         (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
298         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300         output_root: str = ""
301         local: dict[str, Any] = Field(default_factory=dict)
```

```
        s3: dict[str, Any] = Field(default_factory=dict)
        gcs: dict[str, Any] = Field(default_factory=dict)
        gdrive: dict[str, Any] = Field(default_factory=dict)


    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)
        storage: StorageConfig = Field(default_factory=StorageConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain
dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator
on
        # real runs. Scalar leaves are returned as-is.
        def get(self, key: str, default: Any = None) -> Any:
            _MISSING = object()
            val: Any = _MISSING
            if key in type(self).model_fields or hasattr(self, key):
                val = getattr(self, key, _MISSING)
            if val is _MISSING:
                # Legacy convenience: find a leaf key inside any nested section. Sections
are
                # discovered dynamically from the model fields, so a future top-level
section
                # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val
```

```
359
360         def __getitem__(self, key: str) -> Any:
361             val = self.get(key)
362             if val is None and key not in type(self).model_fields:
363                 # Re-raise like a real dict for unknown top-level
364                 raise KeyError(key)
365             return val
366
367
368     # Convenience type alias for callers that want to annotate
369     Config = AppConfig
370
```

20. user (2026-06-20T08:59:52.325Z)

```
total 48
drwxr-xr-x 1 avidu 197609     0 Jun 11 20:44 .
drwxr-xr-x 1 avidu 197609     0 Jun 19 13:03 ..
-rw-r--r-- 1 avidu 197609 16350 Jun 11 20:44 app.js
-rw-r--r-- 1 avidu 197609  8743 Jun  7 14:14 index.html
-rw-r--r-- 1 avidu 197609 13418 Jun  7 14:14 style.css
---
__init__.py
__pycache__
base.py
gcs.py
gdrive.py
local.py
ref.py
s3.py
```

21. assistant (2026-06-20T08:59:56.239Z)

There IS a static UI. Let me check what the existing app.js drives versus what's missing, and confirm the gemini segmenter's key requirement. Let me also check the database job methods quickly.

22. user (2026-06-20T08:59:56.987Z)

```
1       // Badminton Highlight & Rally Indexer JS Client
2
3       document.addEventListener("DOMContentLoaded", () => {
4           // --- APP STATE ---
5           let currentVideoId = null;
6           let indexedRallies = [];
7           let selectedRallyIds = new Set();
8           let playerPool = ["Avi", "Suresh", "Ramesh", "Deepak"];
9
10          // --- UI ELEMENTS ---
11          const videoPathInput = document.getElementById("video-path-input");
12          const playerPoolInput = document.getElementById("player-pool-input");
13          const btnProcess = document.getElementById("btn-process-video");
14
15          const validationList = document.getElementById("validation-checker-list");
16          const ralliesResultsList = document.getElementById("rallies-results-list");
17          const rallyCountLabel = document.getElementById("rally-count-label");
18
19          // Query Filters
20          const queryServer = document.getElementById("query-server");
21          const queryReceiver = document.getElementById("query-receiver");
22          const queryDuration = document.getElementById("query-duration");
23          const queryEvent = document.getElementById("query-event");
24          const btnQuery = document.getElementById("btn-run-query");
25
```

```
        // Compiler Bar
        const compileBoard = document.getElementById("highlight-compile-board");
        const compileSelectedCount = document.getElementById("compile-selected-count");
        const highlightOutputName = document.getElementById("highlight-output-name");
        const btnCompile = document.getElementById("btn-compile-highlights");

        // Loading Overlay
        const loadingOverlay = document.getElementById("loading-overlay");
        const loadingTitle = document.getElementById("loading-title");
        const loadingSubtitle = document.getElementById("loading-subtitle");

        // --- HELPERS ---
        const showLoader = (title, subtitle) => {
            loadingTitle.textContent = title;
            loadingSubtitle.textContent = subtitle;
            loadingOverlay.style.display = "flex";
        };

        const hideLoader = () => {
            loadingOverlay.style.display = "none";
        };

        const populatePlayerSelects = (players) => {
            // Clear except first
            queryServer.innerHTML = '<option value="">Any Server</option>';
            queryReceiver.innerHTML = '<option value="">Any Receiver</option>';

            players.forEach(player => {
                const opt1 = document.createElement("option");
                opt1.value = player;
                opt1.textContent = player;
                queryServer.appendChild(opt1);

                const opt2 = document.createElement("option");
                opt2.value = player;
                opt2.textContent = player;
                queryReceiver.appendChild(opt2);
            });
        };

        // Initialize player selects at start
        populatePlayerSelects(playerPool);

        // --- 1. RUN VIDEO PROCESSING & VALIDATIONS ---
        btnProcess.addEventListener("click", async () => {
            const videoPath = videoPathInput.value.trim();
            if (!videoPath) {
                alert("Please enter a valid local MP4 video file path.");
                return;
            }

            // Parse custom players
            const rawPlayers = playerPoolInput.value.trim();
            if (rawPlayers) {
                playerPool = rawPlayers.split(",").map(p => p.trim()).filter(p => p.length >
0);
                populatePlayerSelects(playerPool);
            }

            showLoader("Analyzing Video Stream", "Running structural diagnostics, video
focus checks, and relevance validators...");

            try {
                // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
                const response = await fetch("/api/process", {
```

```
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({
                        video_path: videoPath,
                        player_pool: playerPool
                    })
                });

                if (!response.ok) {
                    const err = await response.json();
                    throw new Error(err.detail || "Server failed to process the request.");
                }

                const submitted = await response.json();
                const jobId = submitted.job_id;

                // Poll until the job reaches a terminal state (done | failed).
                let job;
                for (;;) {
                    const jr = await fetch(`/api/jobs/${jobId}`);
                    if (!jr.ok) {
                        const err = await jr.json();
                        throw new Error(err.detail || "Failed to poll job state.");
                    }
                    job = await jr.json();
                    if (job.status === "done" || job.status === "failed") break;
                    showLoader("Analyzing Video Stream",
                        `Job ${job.status}${job.progress ? " ? " + job.progress : ""}?`);
                    await new Promise(res => setTimeout(res, 3000));
                }

                if (job.status === "failed") {
                    throw new Error(job.error || "Processing job failed.");
                }

                const data = job.result || {};
                currentVideoId = data.video_id;

                // Render validation list
                renderValidationResults(data.results);

                if (data.status === "failed") {
                    hideLoader();
                    alert("Validation Failed! This video does not meet our quality or
structural criteria. Please review the checklist.");
                    renderEmptyRallies("Validation failed. Please load a valid badminton raw
video.");
                    compileBoard.style.display = "none";
                    return;
                }

                // Successfully processed (field is play_segments ? the old `rallies` read
was a bug)
                indexedRallies = data.play_segments || [];
                selectedRallyIds.clear();
                renderRallyCards(indexedRallies);

                hideLoader();
                alert(`Success! Successfully processed and indexed ${indexedRallies.length}
badminton rallies.`);

            } catch (error) {
                hideLoader();
                alert(`Error: ${error.message}`);
                renderEmptyRallies("Error processing file.");
```

```
            }
        });

        // --- 2. RENDER VALIDATION CHECKLIST ---
        const renderValidationResults = (results) => {
            validationList.innerHTML = "";

            results.forEach(res => {
                const div = document.createElement("div");
                div.className = `validation-item ${res.passed ? 'passed' : 'failed'}`;

                const icon = document.createElement("div");
                icon.className = "val-icon";
                icon.textContent = res.passed ? "?" : "?";

                const info = document.createElement("div");
                info.className = "val-info";

                const title = document.createElement("h4");
                title.textContent = formatValidatorName(res.validator_name);

                const desc = document.createElement("p");
                desc.textContent = res.message;

                info.appendChild(title);
                info.appendChild(desc);

                // If there are detailed metrics, render them
                if (res.details && Object.keys(res.details).length > 0) {
                    const detailsSpan = document.createElement("span");
                    detailsSpan.className = "val-details";
                    detailsSpan.textContent = formatDetails(res.details);
                    info.appendChild(detailsSpan);
                }

                div.appendChild(icon);
                div.appendChild(info);
                validationList.appendChild(div);
            });
        };

        const formatValidatorName = (name) => {
            return name.split("_").map(w => w.charAt(0).toUpperCase() + w.slice(1)).join("
");
        };

        const formatDetails = (details) => {
            return Object.entries(details)
                .map(([k, v]) => `${k}: ${v}`)
                .join(" | ");
        };

        // --- 3. RENDER RALLY TIMELINE CARDS ---
        const renderRallyCards = (rallies) => {
            ralliesResultsList.innerHTML = "";
            selectedRallyIds.clear();
            updateCompilerBar();

            rallyCountLabel.textContent = `${rallies.length} Rallies Found`;

            if (!rallies || rallies.length === 0) {
                renderEmptyRallies("No rallies found matching your filters.");
                return;
            }

```

```
            rallies.forEach(rally => {
                const card = document.createElement("div");
                card.className = `rally-card ${selectedRallyIds.has(rally.id) ? 'selected' :
''}`;
                card.setAttribute("id", `rally-card-${rally.id}`);

                const meta = document.createElement("div");
                meta.className = "rally-meta";

                const title = document.createElement("h3");
                title.textContent = `Rally #${rally.id} (Serve: ${rally.server} ? Receive:
${rally.receiver})`;

                const sub = document.createElement("p");
                sub.innerHTML = `
                    <span><strong>Start:</strong> ${formatTime(rally.start_time)}</span>
                    <span><strong>End:</strong> ${formatTime(rally.end_time)}</span>
                `;

                const tagsDiv = document.createElement("div");
                tagsDiv.className = "rally-tags";

                // Long rally tag
                if (rally.duration > 10) {
                    const longTag = document.createElement("span");
                    longTag.className = "rally-tag long";
                    longTag.textContent = "Long Rally";
                    tagsDiv.appendChild(longTag);
                }

                // Check events to see if we have smashes or fouls
                const events = rally.events || [];
                const hasSmash = events.some(e => e.type === "smash");
                const hasFoul = events.some(e => e.type === "foul");

                if (hasSmash) {
                    const smashTag = document.createElement("span");
                    smashTag.className = "rally-tag smash";
                    smashTag.textContent = "Contains Smash";
                    tagsDiv.appendChild(smashTag);
                }

                if (hasFoul) {
                    const foulTag = document.createElement("span");
                    foulTag.className = "rally-tag foul";
                    foulTag.textContent = "Ends in Foul";
                    tagsDiv.appendChild(foulTag);
                }

                meta.appendChild(title);
                meta.appendChild(sub);
                meta.appendChild(tagsDiv);

                const action = document.createElement("div");
                action.className = "rally-action";

                const dur = document.createElement("div");
                dur.className = "rally-dur";
                dur.textContent = `${rally.duration.toFixed(1)}s`;

                const chk = document.createElement("div");
                chk.className = "checkbox-custom";

                action.appendChild(dur);
                action.appendChild(chk);
```

```
                card.appendChild(meta);
                card.appendChild(action);

                // Card Toggle Selection Click Listener
                card.addEventListener("click", () => {
                    if (selectedRallyIds.has(rally.id)) {
                        selectedRallyIds.delete(rally.id);
                        card.classList.remove("selected");
                    } else {
                        selectedRallyIds.add(rally.id);
                        card.classList.add("selected");
                    }
                    updateCompilerBar();
                });

                ralliesResultsList.appendChild(card);
            });
        };

        const renderEmptyRallies = (msg) => {
            ralliesResultsList.innerHTML = `
                <div class="empty-state">
                    <svg width="48" height="48" fill="none" stroke="currentColor" stroke-
width="1.5" viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" d="M3.75
5.25h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5"/></svg>
                    <h3>No Rallies Available</h3>
                    <p>${msg}</p>
                </div>
            `;
        };

        const formatTime = (seconds) => {
            const m = Math.floor(seconds / 60);
            const s = Math.floor(seconds % 60);
            const ms = Math.floor((seconds % 1) * 10);
            return `${m.toString().padStart(2, '0')}:${s.toString().padStart(2,
'0')}.${ms}`;
        };

        // --- 4. RALLY TIMELINE FILTER QUERY ---
        btnQuery.addEventListener("click", async () => {
            if (!currentVideoId) {
                alert("Please process a video first before running filters.");
                return;
            }

            const server = queryServer.value;
            const receiver = queryReceiver.value;
            const duration = parseFloat(queryDuration.value) || null;
            const event = queryEvent.value;

            showLoader("Filtering Database Index", "Fetching rallies matching exact
relational conditions...");

            try {
                const res = await fetch("/api/query", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({
                        video_id: currentVideoId,
                        server: server || null,
                        receiver: receiver || null,
                        duration_min: duration || null,
                        event_type: event || null
```

```
                    })
                });

                if (!res.ok) throw new Error("Failed to filter index.");

                const data = await res.json();
                renderRallyCards(data.rallies || []);
            } catch (err) {
                alert(`Query failed: ${err.message}`);
            } finally {
                hideLoader();
            }
        });

        // --- 5. COMPILER BOARD MANAGEMENT ---
        const updateCompilerBar = () => {
            const count = selectedRallyIds.size;
            if (count > 0) {
                compileBoard.style.display = "flex";
                compileSelectedCount.textContent = `${count} Rally Segment${count > 1 ? 's'
: ''} Selected`;
            } else {
                compileBoard.style.display = "none";
            }
        };

        btnCompile.addEventListener("click", async () => {
            if (!currentVideoId || selectedRallyIds.size === 0) {
                alert("No rallies selected for compiling highlights.");
                return;
            }

            const outputName = highlightOutputName.value.trim() ||
"highlights_compiled.mp4";
            showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg
stitching demuxers...");

            try {
                const res = await fetch("/api/compile", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({
                        video_id: currentVideoId,
                        // API field is segment_ids (the old rally_ids never matched the
request model)
                        segment_ids: Array.from(selectedRallyIds),
                        output_filename: outputName
                    })
                });

                if (!res.ok) {
                    const err = await res.json();
                    throw new Error(err.detail || "Concatenation failed.");
                }

                await res.json();
                // B3 (PR-2): offer the reel via the streaming download endpoint instead of
                // alerting a server-local path the browser can't reach.
                const dlUrl =
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
                let dl = document.getElementById("highlight-download-link");
                if (!dl) {
                    dl = document.createElement("a");
                    dl.id = "highlight-download-link";
                    dl.style.cssText = "display:block;margin-top:10px;font-weight:bold;";
```

```
398                     compileBoard.appendChild(dl);
399                 }
400                 dl.href = dlUrl;
401                 dl.setAttribute("download", outputName);
402                 dl.textContent = `? Download ${outputName}`;
403                 alert(`Highlight compilation completed! Use the download link to save
${outputName}.`);
404 
405             } catch (error) {
406                 alert(`Stitching compilation failed: ${error.message}`);
407             } finally {
408                 hideLoader();
409             }
410         });
411     });
412 
```

23. assistant (2026-06-20T08:59:59.795Z)

Now I have a full picture of the existing UI. Let me confirm the gemini segmenter's API key requirement and check the database job-related methods to characterize crash recovery and concurrency.

24. user (2026-06-20T09:00:01.844Z)

```
80 
81              # Get AI provider and model config
82              idx_cfg = self.config.get("indexing", {})
83              provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
84 
85              # Get appropriate API key based on the provider
86              api_key_env_var = f"{provider_name.upper()}_API_KEY"
87              api_key = os.environ.get(api_key_env_var)
88              if not api_key:
89                  from backend.pipeline.segmenters._keyhint import api_key_hint
90                  logger.error(
91                      f"{api_key_env_var} environment variable is not set! "
92                      f"To run the {provider_name} segmenter, set your API key. "
93                      + api_key_hint(api_key_env_var)
94                  )
95                  return None
96 
97              try:
98                  provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
99              except KeyError as e:
100                 logger.error(str(e))
101                 return None
102 
103             return sport_profile, provider
104 
105         def _maybe_trim_video(self, video_id: str, video_path: str) -> Tuple[str,
Callable[[], None]]:
106             """Optionally trim the video to indexing.debug_duration seconds (debug feature).
107 
108             Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
109             temp trimmed file created here.
110             """
111             # A: Check for debug_duration to trim the video first
112             debug_duration = self.config.get("indexing", {}).get("debug_duration")
113             video_to_upload = video_path
114             is_trimmed = False
115             temp_trimmed_path = None
```

```
116
117             if debug_duration:
118                 logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
119                 temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
120                 os.makedirs("output", exist_ok=True)
121                 if os.path.exists(temp_trimmed_path):
122                     try:
123                         os.remove(temp_trimmed_path)
124                     except Exception:
125                         pass
126
127                 cmd = [
128                     "ffmpeg", "-y",
129                     "-ss", "0",
130                     "-t", str(debug_duration),
131                     "-i", video_path,
132                     "-c", "copy",
133                     temp_trimmed_path
134                 ]
135                 logger.debug(f"Running command: {' '.join(cmd)}")
136                 try:
137                     subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
138                     logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
139                     video_to_upload = temp_trimmed_path
```

25. user (2026-06-20T09:00:03.407Z)

```
486:    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487-                   player_pool: Optional[List[str]] = None,
488-                   max_frames: Optional[int] = None,
489-                   policy: Optional[Dict[str, Any]] = None) -> bool:
490-        """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
491-        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492-        return self._execute_write(
493-            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy) "
494-            "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495-            (job_id, video_path, segmenter,
496-             json.dumps(player_pool) if player_pool is not None else None,
497-             max_frames, json.dumps(policy) if policy is not None else None),
498-            f"Failed to create job {job_id}",
--
520:    def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521-        """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522-        validation) is inside `result` ? job status 'done' means 'the worker finished'."""
523-        return self._execute_write(
524-            "UPDATE jobs SET status='done', progress='finished', result=?, "
525-            "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
526-            (json.dumps(result), job_id),
527-            f"Failed to mark job {job_id} done",
528-        )
529-
530:    def mark_job_failed(self, job_id: str, error: str) -> bool:
531-        """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
532-        return self._execute_write(
533-            "UPDATE jobs SET status='failed', error=?, "
534-            "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
535-            f"Failed to mark job {job_id} failed",
536-        )
537-
538-    def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
```

```
539-        """Fetch one job; `result`/`player_pool` JSON deserialized."""
540-        with self._connect() as conn:
541-            row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
542-            if not row:
--
559:    def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] = None)
-> bool:
560-        """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now, releasing
the
561-        worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
562-        computed in SQLite's own UTC format so it compares directly to CURRENT_TIMESTAMP."""
563-        return self._execute_write(
564-            "UPDATE jobs SET status='waiting', "
565-            "next_poll_at=datetime('now', ?), "
566-            "progress=COALESCE(?, progress) WHERE id = ?",
567-            (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
568-            f"Failed to set job {job_id} waiting",
569-        )
570-
571:    def seconds_until_next_due(self) -> Optional[float]:
572-        """Seconds until the soonest parked ('waiting') job becomes due, so the worker can
573-        schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one is
574-        already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
575-        try:
576-            with self._connect() as conn:
577-                row = conn.cursor().execute(
578-                    "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579-                    "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS secs "
580-                    "FROM jobs WHERE status='waiting'").fetchone()
581-            if row is None or row["secs"] is None:
582-                return None
583-            return max(0.0, float(row["secs"]))
--
588:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
589-        """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose ``next_poll_at``
is due ?
590-        flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591-        double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592-        first. Returns the claimed job dict, or None if nothing is runnable."""
593-        try:
594-            with self._connect() as conn:
595-                cur = conn.cursor()
596-                row = cur.execute(
597-                    "SELECT id, status FROM jobs WHERE status='queued' "
598-                    "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599-                    "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1").fetchone()
600-                if not row:
--
613:    def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614-        """Crash recovery: any job still queued/running from a previous process is dead
615-        (the executor is in-process). Mark failed so clients see a terminal state, not a
616-        hang. Called once at startup. Returns the number of jobs swept."""
617-        try:
618-            with self._connect() as conn:
619-                cur = conn.cursor()
620-                cur.execute(
621-                    "UPDATE jobs SET status='failed', error=?, finished_at=CURRENT_TIMESTAMP
"
622-                    "WHERE status IN ('queued', 'running')", (reason,))
```

```
623-                conn.commit()
624-                return cur.rowcount
625-        except Exception as e:
```

26. assistant (2026-06-20T09:00:06.872Z)

Note one important detail: `fail_stale_jobs` sweeps both `queued` and `running` on restart, but NOT `waiting`. Let me verify that and check whether the highlights download / job listing endpoints exist (a key UI gap is listing all videos/jobs). Let me grep the endpoints.

27. user (2026-06-20T09:00:08.954Z)

```
backend\main.py:52:@app.get("/", response_class=HTMLResponse)
backend\main.py:121:@app.get("/api/config")
backend\main.py:240:@app.post("/api/process", status_code=202)
backend\main.py:274:@app.get("/api/jobs/{job_id}")
backend\main.py:307:@app.get("/api/highlights/{video_id}/{filename}")
backend\main.py:331:@app.post("/api/query")
backend\main.py:342:@app.post("/api/compile")
backend\api\uploads.py:64:@router.post("/api/upload/init")
backend\api\uploads.py:89:@router.put("/api/upload/{upload_id}/part/{index}")
backend\api\uploads.py:123:@router.post("/api/upload/{upload_id}/complete")
backend\api\uploads.py:147:@router.delete("/api/upload/{upload_id}")
```

28. user (2026-06-20T09:00:09.690Z)

```
588         def claim_due_job(self) -> Optional[Dict[str, Any]]:
589             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
590             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592             first. Returns the claimed job dict, or None if nothing is runnable."""
593             try:
594                 with self._connect() as conn:
595                     cur = conn.cursor()
596                     row = cur.execute(
597                         "SELECT id, status FROM jobs WHERE status='queued' "
598                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600                     if not row:
601                         return None
602                     cur.execute(
603                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                         "WHERE id=? AND status=?", (row["id"], row["status"]))
605                     conn.commit()
606                     if cur.rowcount != 1:
607                         return None                     # lost the race to a concurrent
worker
608                 return self.get_job(row["id"])
609             except Exception as e:
610                 logger.error(f"Failed to claim a due job: {e}")
611                 return None
612
613         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614             """Crash recovery: any job still queued/running from a previous process is dead
615             (the executor is in-process). Mark failed so clients see a terminal state, not a
616             hang. Called once at startup. Returns the number of jobs swept."""
617             try:
```

29. assistant (2026-06-20T09:00:16.686Z)

I now have the complete picture. The full endpoint list is exactly 11 routes across main.py + uploads.py. I have confirmed:
- The full upload?process?index?reel flow
- The in-process job worker behavior (single-box, max_workers=1, DB-as-clock, crash recovery sweeps queued/running but NOT waiting)
- Segmenter/detector selection (config default `gemini`; trajectory hybrids resolve a runner via detector_impl/RALLY_DETECTOR_IMPL: auto=WSL+GPU, native=GPU box, stub=CPU)
- Requirements per segmenter (Gemini key vs GPU vs stub vs skip_ai_handoff)
- The existing static UI (index.html/app.js) and its gaps

Let me record the StructuredOutput.

30. user (2026-06-20T09:01:40.217Z)

Structured output provided successfully

31. assistant (2026-06-20T09:01:51.219Z)

Mapped the full FastAPI loop. Summary of what I returned via StructuredOutput:

- **Process flow**: upload (optional chunked) ? POST /api/process (202 + job, all slow work incl. MD5 on worker) ? worker `_execute_processing` (hash?validate?segment?report, destroy-after-confirm) ? poll /api/jobs/{id} every 3s ? /api/query ? /api/compile ? /api/highlights download.
- **Endpoints**: 11 total (7 in main.py, 4 in uploads.py), each with path:line, method, req/resp.
- **Job worker**: single in-process ThreadPoolExecutor max_workers=1, DB-as-clock (no central scheduler), atomic claim_due_job, JobWaiting/WAIT_AND_RESUME hook (dormant in prod), crash recovery sweeps queued/running?failed at startup (caveat: NOT 'waiting').
- **Segmenter/detector selection**: config.json default = `gemini` (needs GEMINI_API_KEY, no GPU); trajectory hybrids pick a runner via RALLY_DETECTOR_IMPL/detector_impl ? auto=WSL+GPU, native=GPU box (M3.5, next build), stub=CPU mock; skip_ai_handoff drops the Gemini stage. Only `motion` is real+local+keyless.
- **UI gaps**: a static SPA already drives the core index?reel loop for a server-local path, but browser upload isn't wired, there are no list-videos/list-jobs/list-reels endpoints, no segmenter/config picker, a /api/query field mismatch (`rallies` vs `play_segments`), poll-only coarse progress, no auth/multi-tenant, and the decoupled storage/worker path is invisible to the UI.